

Browse Posts Page

This task intends to consolidate all your previous knowledge acquired on React States, React Hooks, Firebase and React Routes. You will expand your DEV@Deakin web platform to be highly dynamic and interactive.

Complete the following:

1. Complete the Task Requirements.
2. Create a new [GitLab](#) repository for this task. Ensure all files/resources created are committed to this repository.
3. Create a 3 – 6 minute video where you provide a walkthrough of your code and solution, demonstrating your understanding of the concepts. Upload your video to [Panopto](#).
4. Create a short report that includes:
 - a. Screenshots of your DEV@Deakin web platform.
 - b. Screenshots of your Firestore of saved posts.
 - c. Screenshots of your Firestore of saved users.
 - d. A link to your repository on GitLab.
 - e. A link to your video.
5. **Upload** to OnTrack and **engage with feedback** to get this complete!

Task Requirements

Before completing this task, ensure you have completed all previous Pass, Credit, and Distinction tasks, and that your DEV@Deakin web platform has all the previous requirements and design criteria implemented successfully.

1. GitLab repository

- Create a New Project called Task D2. Set the Visibility Level to **Private**.

- Once the project is created, under **Manage->Members** add the Unit Chair (Aaron Spence) and your OnTrack marking tutor **as a Reporter role**.
- Strictly do not submit your node_modules folder to your repository. To prevent this, create a .gitignore file, and add inside it node_modules/ to prevent this folder from being tracked by Git.
- Commit all code and resources (e.g. images) that you develop for this task into this repository, with proper commit comments.

Failure to follow the above steps means your task cannot be marked as Complete.

2. Storing Posts

In Task C1 we implemented the Post page, allowing users to input details in a form and submit a Question or Article. Expand the form like so:

New Post

Select Post Type: ☐ Question ☒ Article

Select Post Plan: ☐ Free ☒ Paid

What do you want to ask or share

This section is designed based on the type of the post. It could be developed by conditional rendering. For an article, the following section would be appeared.

Title

Abstract

Article Text

Tags

Post

Building on our work in Task P4 where we integrated Firebase into our DEV@Deakin web platform, enable the Post button when clicked to save all input form details into your Firestore, with the following criteria:

- A user must first be logged in to submit a post.
- Implement proper data validation on all data entered by the user, before allowing for a post to be considered successful.
- Store alongside each post the user's ID, and the date created.
- Alert the user of the outcome, whether successful or an error.

Important: Because we require users to be logged in to make a post, you must refactor your code such that storing posts in your Firestore is now handled in the backend instead of your frontend.

3. Browse Posts Page

Expand your navigation bar to include a Browse Post link. On your Browse Posts page, display a list of all previous posts made by users (e.g. from your Post page). It is up to you how you wish to present this information, but ensure it is displayed with user-experience in mind.

Your Browse Posts page should only show posts that align with the current user's subscription plan:

- A visitor on your site who is not currently logged in can only see free Questions/Articles.
- A logged in user on a Free plan can only see free Questions/Articles.
- A logged in user on a Paid plan can see both free and paid Questions/Articles.

Determination of which posts a user has access to and sees on their interface should be determined by your backend, not your frontend. Your frontend should be sending a HTTP request to the backend, requesting the right set of posts to

fetch. That is: you should not be fetching posts from your Firestore from the frontend directly – they should be served from the backend instead.

For example, if the user is on a Free plan, the paid-only Questions/Articles should not be stored/present on your client-side frontend at all, even in the code.

Have a think about why in this scenario all posts should be fetched from your backend, instead of your frontend. Feel free to discuss this topic on the SIT313 Teams channel.

4. Browse Posts Filtering

Your Browse Posts page should allow the following user features:

- Filtering – Can filter posts based on fields such as created date, post type (article/question), post plan (paid/free), tags, and so on.
- Hide – The user can hide any post from their list. Note that it should not remove the post from the Firestore.
- Reset – Reset the page back to its original state (e.g. remove any filtering or hiding).
- Expand – In your list, only a preview of the whole Article/Question should be shown. To display the entirety of the Post the user should be able to click on it to expand to a larger view.
- Draggable (optional) – Allow the user to re-order the list by dragging each post. See the **Optional NPM Packages** section.

5. Testing and Validation

The above Task Requirements should each be easily testable with the following test cases:

- Can login and logout of different users.
- Can visibly see if a user is on a Free or Paid plan.
- Can upgrade a user from a Free to a Paid plan.

- Can visit the Browse Posts page, which dynamically adjusts depending on the user that is logged in.
- Any user input data (when creating a post, or upgrading their plan) should be valid before being able to proceed.

Ensure to capture in your recorded video each of the above test cases.

Optional NPM Packages

There is a rich ecosystem of amazing packages to enhance our development. Here are some that may assist you:

- [React-Draggable](#) – Allows elements to be draggable.
- [React-Markdown](#) – A React component to render Markdown.
- [SweetAlert2](#) – Beautiful, responsive, easy-to-use user popups.
- [React-Toastify](#) – Nice and easy user notifications.

Caution: Beware of malicious NPM packages!

Unfortunately, NPM packages are now an active area infiltrated by malicious actors. NPM packages may appear 'safe' at glance, but may have embedded malicious code. Before installing any NPM package, investigate whether it is from a trustworthy author(s).

You are responsible for your own cyber-safety.

Available Resources

We have many resources to assist in your learning journey – if you are ever unsure then please reach out to us, we are always happy to help.

- **The SIT313 Teams channel** – an active learning resource with weekly announcements from the Unit Chair, and regular discussions/assistance from the SIT313 teaching team and students.

- **Seminars** – you can attend both on-campus and online sessions, and join as many as you like each week. Your tutors will provide demonstrations and assistance to help guide your learning.
- **SIT313 CloudDeakin** – contains a large collection of text and video resources.
- **Announcements** - Keep an eye on your email and any announcements made in the SIT313 Teams channel.

Caution: Acceptable use of Artificial Intelligence

It is acceptable to use AI as a learning tool. However, you can **never** ask AI to write code to complete some/parts of this task for you: this is cheating because it does not demonstrate that you understand anything. Detection of inappropriate use of AI has a very real consequences, such as failing the task, or even failing the entire unit.